PROYECTO INTEGRADOR

Sistema de Administración de un Club Deportivo

Análisis, diseño, desarrollo, prueba y puesta a punto

Tecnología: C# · Windows Forms · MySQL

Arquitectura en capas: Presentación · Negocio · Datos · Entidades

Índice

Índice **2**

1. Enunciado del requerimiento del cliente **4**

1.1 Reglas de negocio relevadas 4

1.2 El sistema debe permitir controlar 4

1.3 Entregables solicitados 4

2. Estrategia de solución **5**

2.1 Ciclo de vida aplicado (Fases 1 a 7) 5

3. Lista de variables **6**

Persona (clase base abstracta) 6

Socio 7

Cuota 7

Profesor / Actividad / Sueldo / AsistenciaProfesor 7

ConsultaNutricion / Usuario 8

Fase 1 — Análisis preliminar **8**

Identificación de clases y atributos 8

Salidas del sistema 8

Entradas de datos 8

Límite y objetivo del sistema 9

Módulos del sistema (desagregación del objetivo) 9

Fase 2 — Análisis previo y funcional **9**

Procesos del sistema 9

Clases con atributos que intervienen 10

Redacción lógica de cada proceso 10

Cobrar cuota (proceso central) 10

Iniciar sesión 10

Alta de socio 10

Registro de no socio e inscripción a la actividad 10

Emisión de carnet 10

Listado diario de vencimientos 11

Gestión de actividades 11

Alta de profesor 11

Firma de asistencia del profesor 11

Liquidación de sueldo 11

Confección de rutinas 11

Asignación de turno de nutrición 11

Reportes 11

Boceto de pantallas de la interfaz 12

Fase 3 — Análisis orgánico **14**

Diálogo del sistema con el usuario 14

Cómo se informan errores de datos 15

Plantillas de casos de uso 15

CU-01 — Iniciar Sesión 15

CU-02 — Registrar Socio 16

CU-03 — Registrar No Socio 16

CU-04 — Inscribir a Actividad 17

CU-05 — Cobrar Cuota 17

CU-06 — Emitir Carnet 18

CU-07 — Listar Vencimientos 18

CU-08 — Gestionar Actividades 18

CU-09 — Confeccionar Rutina 19

CU-10 — Firmar Asistencia 19
CU-11 — Liquidar Sueldo 19
CU-12 — Asignar Turno Nutrición 20
CU-13 — Generar Reportes 20
Fase 4 — Conectar datos para programar 20
Cómo se enlazan los datos con el código 20
Control de acceso de usuarios 22
Base de datos creada en MySQL 22
Fase 5 — Programar 23
Codificación de pantallas 23
Orden de prioridad de programación 23
Acceso a la ventana principal 24
Alta de nuevos socios 24
Consultas a la base de datos 24
Datos cargados directamente en la base 25
Fase 6 — Prueba 25
Quién prueba el sistema 25
Cómo reiniciar el sistema si hay errores 25
Fase 7 — Puesta a punto 26
Cómo crear el ejecutable (Release) 26
Cómo se implementa el sistema 26
4. Diagrama de Clases 27
Lectura del diagrama 27
5. Diagrama de Casos de Uso 28
5.1 Diagrama de Secuencia — Cobrar Cuota (opcional) 29
5.2 Diagrama de Actividades — Alta de Socio (opcional) 29
6. Juego de prueba 30
7. Conclusiones del desarrollo 31
8. Instrucciones paso a paso 31
8.1 Crear el proyecto en Visual Studio 31
8.2 Crear la carpeta «Datos» dentro del proyecto 31
8.3 Instalar el conector MySQL (NuGet MySql.Data) 32
8.4 Configurar la base de datos 32
8.5 Ejecutar y compilar 32
8.6 Generar el ejecutable (Release) y desplegarlo 32

1. Enunciado del requerimiento del cliente

La empresa administra un club deportivo que ofrece actividades tanto para socios como para no socios, y solicita un sistema de escritorio (Windows Forms) con base de datos MySQL que informatice la gestión integral del club. A continuación, se transcribe, organizado, el enunciado relevado del cliente.

1.1 Reglas de negocio relevadas

- Los socios abonan una cuota mensual; los no socios abonan únicamente el costo de la actividad que realizan en el momento, según cartilla de precios.
- La cuota se abona por adelantado, en efectivo o con tarjeta de crédito. Con tarjeta existen promociones en 3 y 6 cuotas.
- La inscripción por primera vez al gimnasio implica completar una ficha y presentar el apto físico.
- El club posee un plantel propio de profesores y un listado de suplentes.
- Vencido el período de pago, el socio automáticamente no puede realizar actividades. Para retomar debe abonar la cuota; el nuevo plazo comienza a correr desde el día siguiente al vencimiento.
- Cada profesor, además de dictar su clase, controla el salón general (musculación y aparatos) y confecciona rutinas para sus alumnos.
- Los profesores firman planilla de asistencia al llegar al club.
- El sueldo de los profesores se liquida mensualmente, el último día hábil del mes.
- El servicio de nutrición funciona una vez por semana, con turno asignado con anticipación; se lleva ficha médica de cada socio y se establece la carga de actividad física permitida.
- Se entrega un carnet al socio.

Supuestos adoptados por el grupo donde el enunciado no fija un valor concreto: el servicio de nutrición atiende los días miércoles con un máximo de 12 turnos por día, y la liquidación de sueldos fuera del último día hábil se permite con un aviso (no se bloquea).

1.2 El sistema debe permitir controlar

1. Registro de socios y no socios.
2. Entrega de carnet y cobro de cuota mensual o diaria.
3. Listado diario de los socios a quienes, en la fecha, les vence la cuota.

1.3 Entregables solicitados

- Enunciado del requerimiento y estrategia de solución.
- Lista de variables con su significado.
- Diagrama de Clases y Diagramas de Casos de Uso.
- Juego de prueba con resultados esperados.
- Código fuente en C# (Windows Forms + MySQL), organizado en capas, listo para ejecutar.
- Conclusiones del desarrollo. Todas las hojas llevan pie de página con Apellido y Nombre de los autores y numeración X/N en el extremo superior derecho.

2. Estrategia de solución

Se optó por una arquitectura en capas en C# / Windows Forms, que separa responsabilidades y facilita el mantenimiento y las pruebas:

Capa	Carpeta	Responsabilidad
Presentación	Formularios/	14 formularios Windows Forms (ABM, cobros, listados). Validan datos de entrada y delegan toda regla de negocio en la capa de Negocio.
Negocio	Negocio/	Reglas del club: cuota al día, promociones con tarjeta, turno de nutrición semanal, último día hábil, etc. No conoce SQL ni controles de UI.
Datos	Datos/	Clase Conexion (abre/cierra la conexión a MySQL) y una clase DAO por entidad, con consultas parametrizadas y, en operaciones críticas, stored procedures.
Entidades	Entidades/	Clases POCO (Persona abstracta, Socio, Profesor, Cuota, etc.) que representan el modelo de datos en memoria.

La carpeta Datos concentra todo lo relacionado con el entorno de datos, tal como pide la consigna: la clase Conexion (con métodos Abrir() / Cerrar(), bloques try/catch/finally y cadena de conexión configurable desde App.config) y las 13 clases DAO. Todas las consultas están parametrizadas (MySqlParameter / AddWithValue) para evitar inyección SQL, y las operaciones críticas —alta de socio, cobro de cuota, listado de vencimientos y liquidación de sueldo— se resuelven con stored procedures (CommandType.StoredProcedure), mientras que las demás usan CommandType.Text.

Decisiones de diseño relevantes:

- Persona es una clase abstracta de la que heredan Socio, NoSocio y Profesor (evita duplicar nombre, DNI, contacto, etc.). En la base de datos se refleja como una tabla persona con las tablas socio, profesor y no_socio relacionadas por id_persona (clave foránea).
- El estado de la cuota no se guarda como bandera manual: se calcula comparando la fecha de hoy contra el último vencimiento pagado (SocioNegocio.PuedeRealizarActividad), así nunca queda desincronizado.
- Las contraseñas de usuario se guardan como hash SHA-256, nunca en texto plano.
- El acceso está controlado por rol (Administrador, Recepcionista, Profesor, Nutricionista): frmMenuPrincipal arma el menú según el rol del usuario autenticado.

2.1 Ciclo de vida aplicado (Fases 1 a 7)

Se resume a continuación el ciclo de vida completo del proyecto —las siete fases pedidas por la cátedra— y en qué parte de este documento se desarrolla cada una. Las siete fases se presentan como secciones consecutivas (Fase 1 a Fase 7); el Juego de prueba (sección 6) y las Instrucciones paso a paso (sección 8) amplían, respectivamente, las Fases 6 y 7.

Fase	Nombre	Dónde se desarrolla
1	Análisis preliminar	Fase 1 — identificación de clases, entradas/salidas, límite y objetivo del sistema.
2	Análisis previo y funcional	Fase 2 — procesos del sistema, redacción lógica de cada proceso y boceto de las 14 pantallas.
3	Análisis orgánico	Fase 3 — diálogo con el usuario, manejo de errores y las 13 plantillas de casos de uso.
4	Conectar datos para programar	Fase 4 — carpeta Datos, clase Conexion, control de acceso y base de datos en MySQL.
5	Programar	Fase 5 — codificación de los 14 formularios y de las operaciones críticas.
6	Prueba	Fase 6 — quién prueba el sistema y cómo reiniciarlo si hay errores; los 14 casos funcionales están en la sección 6 (Juego de prueba).
7	Puesta a punto	Fase 7 — cómo crear el ejecutable (Release) y cómo se implementa el sistema; el detalle paso a paso está en la sección 8.

3. Lista de variables

Se listan las variables más relevantes del modelo, agrupadas por entidad. Los nombres coinciden con las propiedades de las clases en Entidades/ y con las columnas de la base Proyecto.

Persona (clase base abstracta)

Variable	Tipo	Significado
id	int	Identificador único de la persona (PK de persona).
nombre / apellido	string	Nombre y apellido de la persona.
dni	string	Documento de identidad, único en el sistema.
fechaNacimiento	date	Fecha de nacimiento; usada para calcular la edad.
telefono / direccion / email	string	Datos de contacto.

Socio

Variable	Tipo	Significado
nroSocio	string	Número de socio, único, impreso en el carnet.
fechaAlta	date	Fecha de ingreso al club.
aptoFisico	bool	Indica si presentó el apto físico (requisito para carnet y 1ª vez en el gimnasio).
estado	string	'Activo' / 'Inactivo' (baja lógica).
carnetEntregado	bool	Indica si ya se le emitió el carnet.
ultimoVencimiento	date?	(Calculada) fecha de vencimiento de la última cuota pagada.

Cuota

Variable	Tipo	Significado
mes / anio	int	Período que cubre la cuota.
monto	decimal	Importe abonado.
fechaVencimiento	date	Fecha hasta la cual el socio puede realizar actividades.
fechaPago	date	Fecha real en que se realizó el pago.
formaPago	string	'Efectivo' o 'Tarjeta'.
cuotasTarjeta	int?	3 o 6 si la forma de pago es Tarjeta; null en caso contrario.

Profesor / Actividad / Sueldo / AsistenciaProfesor

Variable	Tipo	Significado
legajo	string	Identificador interno del profesor, único.
tipo (Profesor)	string	'Titular' o 'Suplente'.
horariosAsignados	string	Franjas horarias en las que dicta clase / controla el salón.
costoNoSocio	decimal	Precio de la actividad para quien no es socio (cartilla).
cupoMaximo	int	Cantidad máxima de inscriptos a la actividad.
horaEntrada	time	Hora en que el profesor firmó la asistencia del día.
monto (Sueldo)	decimal	Importe liquidado en el período mes/año.

ConsultaNutricion / Usuario

Variable	Tipo	Significado
turno	int	Número de orden del turno del día (el servicio atiende una vez por semana).
cargaActividadPermitida	string	Actividad física habilitada, definida junto al nutricionista.
nombreUsuario / contraseñaHash	string	Credenciales de acceso; la contraseña se guarda con hash SHA-256.
rol	string	'Administrador' | 'Recepcionista' | 'Profesor' | 'Nutricionista'.

Fase 1 — Análisis preliminar

Identificación de clases y atributos

Siguiendo el método del sistema modelo, en el enunciado se marcaron en amarillo las clases y en verde los atributos:

«La empresa administra un club deportivo que tiene distintos tipos de actividades destinadas a socios y no socios. Algunas características de cómo se organiza el club son: cuota, inscripción y plantel. Vencido el periodo de pago, automáticamente el socio no puede realizar actividades; para retomar, debe pagar la cuota, y el plazo de pago comienza a correr a partir del día siguiente al vencimiento de la cuota. Cada profesor, además de dictar su clase en los horarios asignados, controla las actividades del salón general (musculación y aparatos) y confecciona las rutinas para sus alumnos. Los profesores firman una planilla de asistencia a la llegada al club y el cobro de su sueldo es mensual, el último día hábil del mes. El club cuenta con un servicio adicional de Nutrición, que funciona una vez por semana; la consulta es por turno asignado con anticipación. Se lleva la ficha médica de cada socio y se establece la carga de actividad física permitida. Se desea controlar: el registro de los socios y de los no socios; la entrega de carnet y el cobro de cuota mensual o diaria; y el listado diario de los socios que en la fecha les vence la cuota.»

Resultado: las clases identificadas (Socio, NoSocio, Cuota, Inscripcion, Profesor, Actividad, Rutina, AsistenciaProfesor, Sueldo, ConsultaNutricion y Carnet) y sus atributos se detallan en la Lista de variables (sección 3) y en el Diagrama de Clases (sección 4). En el diseño se agregaron Persona, como clase abstracta para los datos comunes, y Usuario, para el control de acceso.

Salidas del sistema

- Carnet de socio (pantalla + registro en BD).
- Listado diario de vencimientos de cuota.
- Comprobante / confirmación de cobro de cuota.
- Listados: socios activos/inactivos, recaudación del mes, carnets emitidos, ocupación de actividades.
- Planilla de asistencia de profesores.
- Liquidación de sueldos del período.

Entradas de datos

- Datos personales de socios, no socios y profesores.
- Datos de la actividad (nombre, costo, horario, cupo, profesor a cargo).

- Datos del cobro de cuota (monto, forma de pago, cuotas de tarjeta).
- Ficha médica / turno de nutrición.
- Firma de asistencia diaria del profesor.
- Credenciales de acceso (usuario / contraseña).

Límite y objetivo del sistema

El sistema abarca la gestión administrativa del club: altas de personas, cobro de cuotas y de actividades, control de acceso por cuota al día, planteles de profesores, rutinas, asistencia, sueldos y turnos de nutrición. No abarca facturación fiscal (AFIP), pasarela de pago real con tarjeta (se registra la forma de pago, no se procesa la transacción financiera) ni control biométrico de acceso físico. El núcleo pedido por el cliente son los tres puntos de la sección 1.2 (registro de socios y no socios, carnet y cobro de cuota, listado diario de vencimientos); el resto de los módulos son una extensión del grupo sobre el mismo modelo.

Módulos del sistema (desagregación del objetivo)

1. Login y control de acceso por rol
2. Gestión de Socios
3. Gestión de No Socios
4. Gestión de Profesores (titulares y suplentes)
5. Gestión de Actividades
6. Gestión de Cuotas (cobro, formas de pago, promociones)
7. Listado diario de vencimientos
8. Entrega de Carnet
9. Control de acceso a actividades (cuota al día)
10. Gestión de Rutinas
11. Asistencia de Profesores
12. Liquidación de Sueldos
13. Consultas de Nutrición (turnos, ficha médica)
14. Reportes y listados

Fase 2 — Análisis previo y funcional

Procesos del sistema

- Alta de socio (con validación de DNI y N° de socio únicos).
- Cobro de cuota mensual (efectivo / tarjeta, con recálculo del vencimiento).
- Registro de no socio y cobro de la actividad puntual.
- Emisión de carnet (sujeta a apto físico y cuota al día).
- Listado diario de vencimientos.
- Alta de profesor, firma de asistencia y liquidación de sueldo.
- Alta y consulta de rutinas.
- Asignación de turnos de nutrición (solo miércoles).

Clases con atributos que intervienen

Ver el Diagrama de Clases (sección 4) y la Lista de variables (sección 3): Persona, Socio, NoSocio, Profesor, Actividad, Cuota, Inscripcion, Carnet, Rutina, AsistenciaProfesor, Sueldo, ConsultaNutricion, Usuario.

Redacción lógica de cada proceso

Cobrar cuota (proceso central)

1. La recepcionista busca al socio por N° de socio o DNI.
2. El sistema muestra nombre, N° de socio y el último vencimiento pagado.
3. La recepcionista ingresa monto, forma de pago (Efectivo/Tarjeta) y, si es Tarjeta, la promoción (3 o 6 cuotas).
4. CuotaNegocio valida: monto > 0; forma de pago válida; promoción solo 3 o 6 con Tarjeta.
5. CuotaDAO ejecuta sp_CobrarCuota:

```
SI el socio nunca pagó      -> nuevo vencimiento = hoy + 1 mes
SI ya tiene un vencimiento  -> nuevo vencimiento =
      (último vencimiento + 1 día) + 1 mes
```

6. Se inserta la cuota y se marca el socio como 'Activo'.
7. El sistema informa la nueva fecha de vencimiento. Si, aun así, quedó en el pasado (mucho atraso), se avisa que el socio sigue vencido y debe abonar los períodos adeudados.

Iniciar sesión

1. El usuario ingresa nombre de usuario y contraseña en frmLogin.
2. El sistema verifica que ambos campos estén completos.
3. UsuarioNegocio calcula el hash SHA-256 de la contraseña y UsuarioDAO busca un usuario activo con ese nombre y ese hash.
4. SI existe, se abre frmMenuPrincipal con el menú de su rol; SI NO, se muestra «Usuario o contraseña incorrectos.» y se limpia la clave.

Alta de socio

1. La recepcionista ingresa datos personales, N° de socio y apto físico.
2. SocioNegocio valida nombre, apellido, DNI y N° de socio obligatorios.
3. Verifica que no exista otra persona con ese DNI ni otro socio con ese número.
4. SocioDAO ejecuta sp_AltaSocio: inserta persona y socio en una misma transacción (estado Activo, carnet no entregado).
5. El socio aparece en la grilla.

Registro de no socio e inscripción a la actividad

1. La recepcionista ingresa nombre, apellido, DNI y elige la actividad.
2. NoSocioNegocio valida los datos y que la actividad exista.
3. Se registra (o reutiliza) la persona como no socio.
4. Se genera la inscripción del día (tipo NoSocio) con el costo de la actividad según la cartilla de precios.

Emisión de carnet

1. La recepcionista busca al socio por N° de socio o DNI.
2. CarnetNegocio verifica: el socio existe, presentó el apto físico y tiene la cuota al día.

3. SI cumple, se registra el carnet y carnetEntregado = verdadero; SI NO, se informa el motivo y no se emite.

Listado diario de vencimientos

1. El usuario elige una fecha (por defecto, hoy).
2. Se ejecuta sp_ListarVencimientosDia(fecha).
3. Se muestran los socios cuya cuota vigente vence ese día y el total.

Gestión de actividades

1. El administrador carga nombre, tipo, costo para no socios, horario, cupo máximo y profesor a cargo.
2. ActividadNegocio valida: nombre obligatorio, costo >= 0, cupo > 0.
3. Se guarda, modifica o da de baja (con confirmación previa).

Alta de profesor

1. El administrador carga datos personales, legajo, especialidad, tipo (Titular o Suplente) y horarios asignados.
2. ProfesorNegocio valida nombre, apellido y legajo obligatorios.
3. Se guarda; el filtro «Solo suplentes» lista el listado de suplentes.

Firma de asistencia del profesor

1. El profesor elige Firmar asistencia al llegar al club.
2. SI ya firmó hoy, se informa «El profesor ya registró su asistencia hoy.»; SI NO, se registra la fecha y la hora de entrada.

Liquidación de sueldo

1. El administrador elige profesor, mes y año e ingresa el monto.
2. SueldoNegocio valida monto > 0 y avisa si hoy no es el último día hábil del mes.
3. sp_LiquidarSueldo inserta o actualiza el sueldo del período con fecha de pago = hoy.

Confección de rutinas

1. El profesor elige al socio y redacta la rutina.
2. RutinaNegocio valida que la descripción no esté vacía.
3. Se guarda con la fecha y el profesor que la confeccionó.

Asignación de turno de nutrición

1. La nutricionista elige socio, fecha, hora y N° de turno, y carga observaciones y la carga de actividad física permitida.
2. NutricionNegocio valida: la fecha es miércoles, turno > 0 y quedan turnos libres (máximo 12 por día).
3. Se registra el turno y queda en el historial del socio.

Reportes

1. El usuario elige el listado: socios activos/inactivos, recaudación del mes, carnets emitidos u ocupación de actividades.
2. Para la recaudación indica mes y año.
3. Se muestra la grilla con la cantidad de registros encontrados.

Boceto de pantallas de la interfaz

A continuación, se muestra el boceto de cada uno de los 14 formularios detallados en la Fase 5, tal como quedaron diseñados en Visual Studio Community.

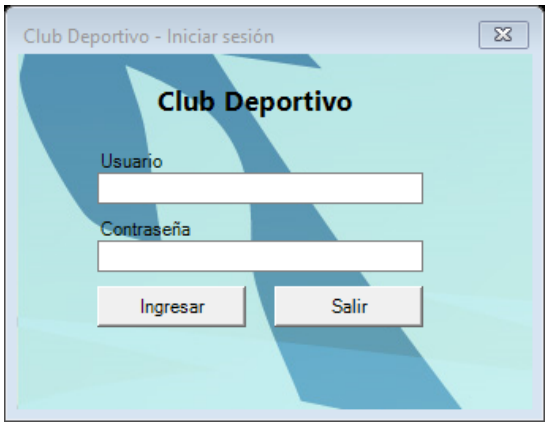


Fig. 1 — frmLogin: Inicio de sesión



Fig. 2 — frmMenuPrincipal: Menú principal (MDI)

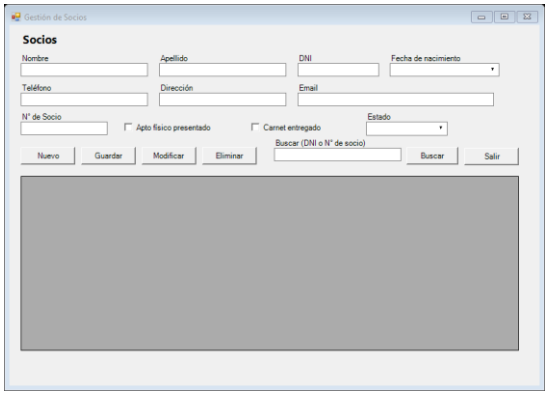


Fig. 3 — frmSocios: Gestión de socios (ABM)

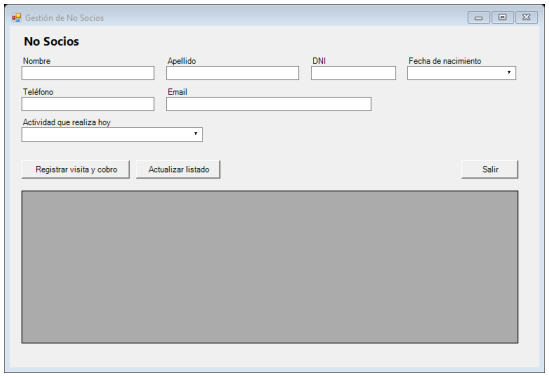


Fig. 4 — frmNoSocios: Gestión de no socios

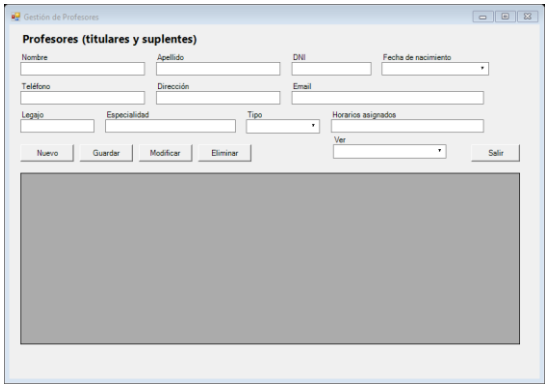


Fig. 5 — frmProfesores: Gestión de profesores

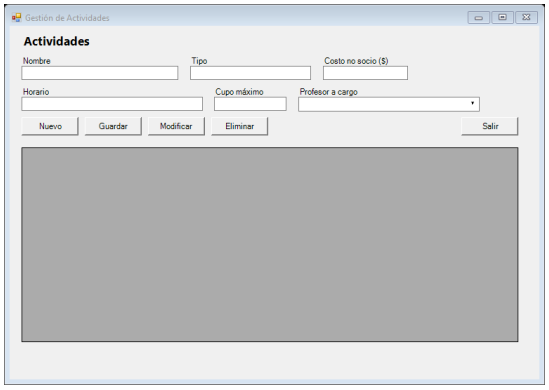


Fig. 6 — frmActividades: Gestión de actividades

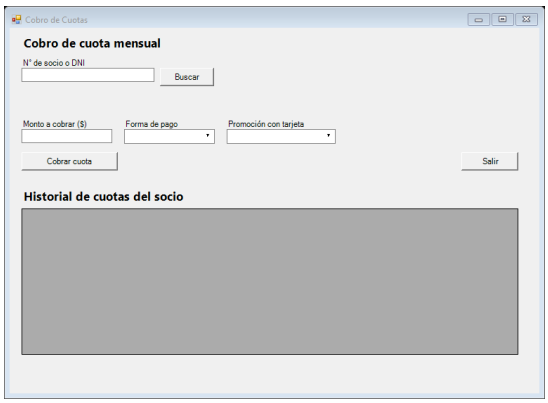


Fig. 7 — frmCuotas: Cobro de cuotas

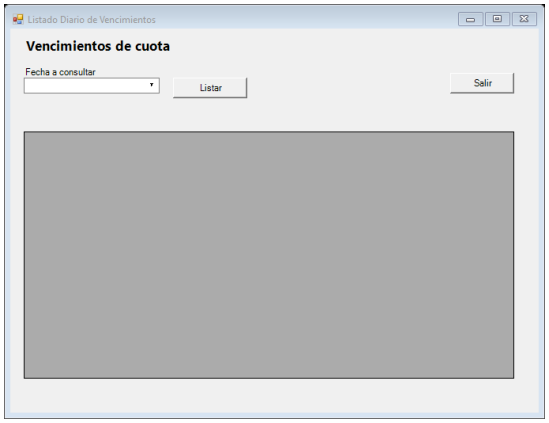


Fig. 8 — frmVencimientos: Listado diario de vencimientos

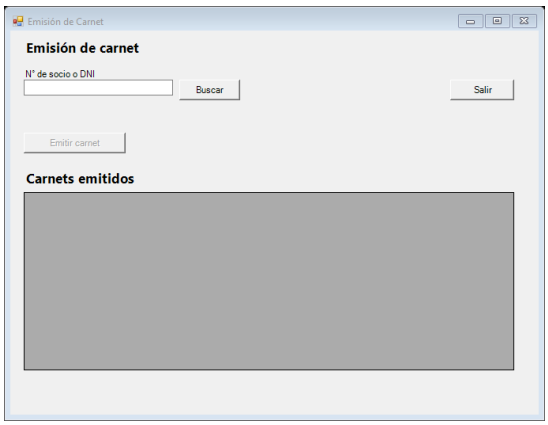


Fig. 9 — frmCarnet: Emisión de carnet

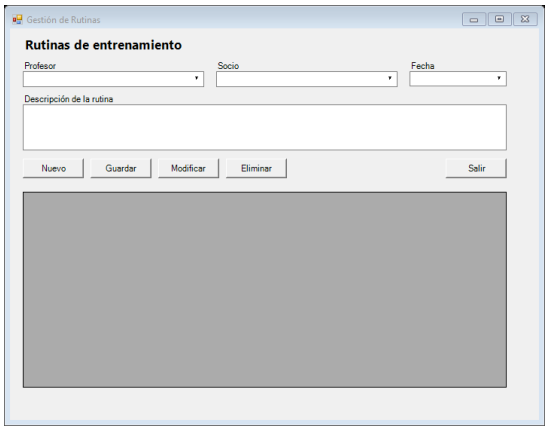


Fig. 10 — frmRutinas: Rutinas

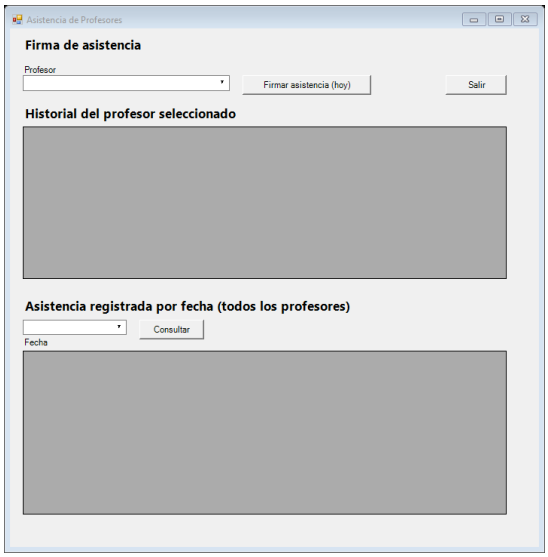


Fig. 11 — frmAsistenciaProfesores: Asistencia de profesores

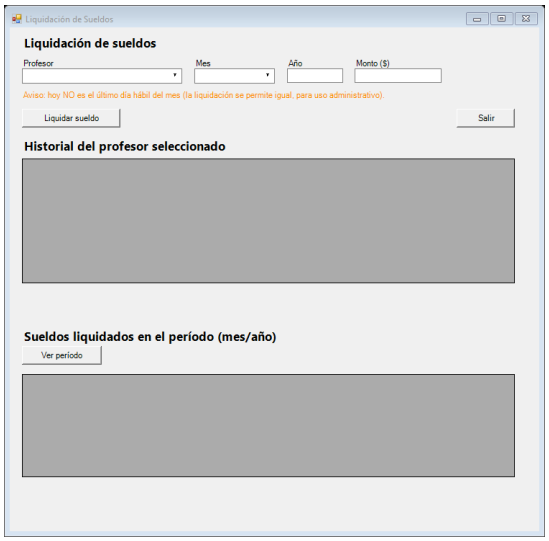


Fig. 12 — frmSueldos: Liquidación de sueldos

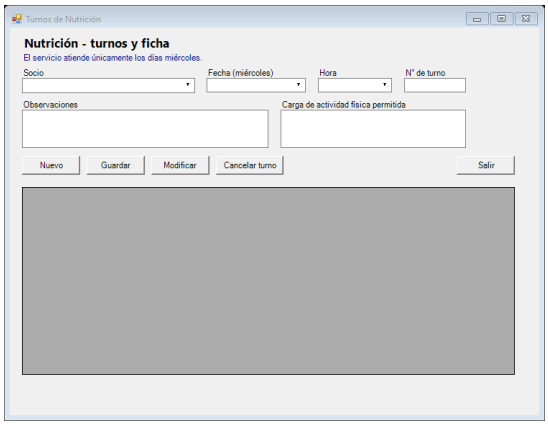


Fig. 13 — frmNutricion: Turnos de nutrición

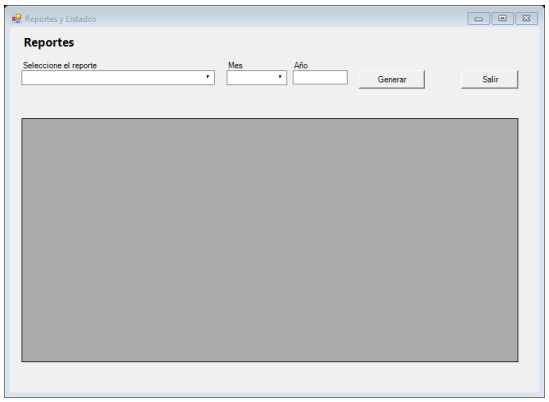


Fig. 14 — frmReportes: Reportes

Fase 3 — Análisis orgánico

Diálogo del sistema con el usuario

- Cada ABM usa siempre los mismos seis botones (Nuevo, Guardar, Modificar, Eliminar, Buscar, Salir) para que el usuario reconozca el patrón en cualquier pantalla.
- Las acciones irreversibles (baja de socio, eliminación de actividad/rutina, cancelación de turno) piden confirmación (UIHelper.Confirmar) antes de ejecutarse.

- Toda operación exitosa muestra un mensaje de confirmación (UIHelper.Info), por ejemplo, la nueva fecha de vencimiento al cobrar una cuota.
- frmMenuPrincipal arma el menú según el rol del usuario autenticado, de modo que cada perfil solo ve las opciones que le corresponden.

Cómo se informan errores de datos

Toda validación pasa por UIHelper antes de tocar la base de datos: Requerido, ComboRequerido, Decimal y Entero muestran un MessageBox de error, ubican el foco en el campo inconsistente y cancelan la operación. Los errores de acceso a datos (conexión caída, violación de una clave única, motor no disponible) se capturan en cada DAO con try/catch/finally, se re-lanzan con un mensaje en español orientado al usuario y la capa de Presentación los muestra sin exponer detalles técnicos de SQL. La clase Conexion cierra siempre la conexión en el finally, incluso si la operación falló.

Plantillas de casos de uso

Para cada proceso del diagrama de casos de uso (sección 5) se redactó su plantilla con el flujo normal y el flujo alternativo, siguiendo el formato del sistema modelo.

CU-01 — Iniciar Sesión

Actor	Administrador, Recepcionista, Profesor, Nutricionista
Precondición	El usuario está registrado y activo en la tabla usuario.
Flujo normal	1. El CU inicia cuando el usuario abre el sistema y se muestra frmLogin. 2. El usuario ingresa nombre de usuario y contraseña. 3. El sistema verifica que ambos datos estén ingresados. 4. El sistema verifica que exista un usuario activo con esas credenciales (hash SHA-256). 5. El sistema abre el menú principal con las opciones de su rol.
Flujo alternativo	A1. Faltan datos: el sistema muestra un mensaje y ubica el foco en el campo vacío. A2. Credenciales incorrectas: el sistema muestra «Usuario o contraseña incorrectos.», limpia la contraseña y permanece en frmLogin.
Postcondición	Sesión iniciada con el rol del usuario.

CU-02 — Registrar Socio

Actor	Recepcionista
Precondición	Sesión iniciada con rol Recepcionista o Administrador.
Flujo normal	1. El CU inicia cuando la recepcionista selecciona Socios > Gestión de socios y presiona Nuevo. 2. Ingresa nombre, apellido, DNI, fecha de nacimiento, contacto, N° de socio y si presentó el apto físico. 3. El sistema verifica que los datos obligatorios estén ingresados. 4. El sistema verifica que no exista otra persona con ese DNI ni otro socio con ese número. 5. El sistema almacena la persona y el socio (sp_AltaSocio) con estado Activo y lo muestra en la grilla.
Flujo alternativo	A1. Faltan datos obligatorios: el sistema muestra un mensaje y cancela la operación. A2. El DNI ya existe: el sistema rechaza el alta y muestra «Ya existe una persona registrada con ese DNI.» A3. El N° de socio ya existe: el sistema rechaza el alta y muestra «Ya existe un socio con ese número.»
Postcondición	Socio registrado, habilitado para el cobro de la primera cuota.

CU-03 — Registrar No Socio

Actor	Recepcionista
Precondición	Sesión iniciada y al menos una actividad cargada.
Flujo normal	1. El CU inicia cuando la recepcionista selecciona Socios > Gestión de no socios. 2. Ingresa nombre, apellido y DNI y selecciona la actividad que la persona realizará. 3. El sistema verifica que los datos obligatorios estén ingresados y que la actividad sea válida. 4. El sistema registra (o reutiliza) la persona como no socio e incluye el CU-04 Inscribir a Actividad. 5. El sistema informa el importe a cobrar según la cartilla de precios.
Flujo alternativo	A1. Faltan nombre, apellido o DNI: el sistema muestra «Nombre y apellido son obligatorios.» o «El DNI es obligatorio.» A2. No se eligió una actividad válida: el sistema muestra «Debe seleccionar una actividad válida.»
Postcondición	Visita del no socio registrada.

CU-04 — Inscribir a Actividad

Actor	Recepcionista
Precondición	Persona registrada como no socio (CU-03) y actividad existente.
Flujo normal	1. El CU se inicia desde el CU-03 al confirmar la visita. 2. El sistema toma el costo para no socios de la actividad elegida. 3. El sistema registra la inscripción con fecha de hoy, tipo NoSocio y el monto pagado.
Flujo alternativo	A1. La actividad no existe: el sistema rechaza la inscripción y muestra un mensaje.
Postcondición	Inscripción registrada. Los socios no se inscriben con pago: acceden a las actividades mientras tengan la cuota al día.

CU-05 — Cobrar Cuota

Actor	Recepcionista
Precondición	Socio registrado.
Flujo normal	1. El CU inicia cuando la recepcionista selecciona Cuotas > Cobro de cuota. 2. Busca al socio por N° de socio o DNI. 3. El sistema muestra los datos del socio, su último vencimiento y el historial de cuotas. 4. Ingresa el monto, la forma de pago y, si es tarjeta, la promoción (3 o 6 cuotas). 5. El sistema verifica que el monto sea mayor a cero y que la forma de pago sea válida. 6. El sistema registra la cuota (sp_CobrarCuota) y muestra el nuevo vencimiento: día siguiente al vencimiento anterior + 1 mes, u hoy + 1 mes si es el primer pago.
Flujo alternativo	A1. El socio no existe: el sistema muestra un mensaje y no habilita el cobro. A2. Monto o forma de pago inválidos: el sistema muestra el error y no registra la cuota. A3. Tarjeta sin promoción válida: «Con tarjeta, la promoción admitida es en 3 o 6 cuotas.» A4. Por atraso, el nuevo vencimiento sigue en el pasado: el sistema avisa que el socio continúa vencido.
Postcondición	Cuota registrada y socio en estado Activo.

CU-06 — Emitir Carnet

Actor	Recepcionista
Precondición	Socio registrado.
Flujo normal	1. El CU inicia cuando la recepcionista selecciona Socios > Emisión de carnet. 2. Busca al socio por N° de socio o DNI. 3. El sistema verifica que el socio exista, haya presentado el apto físico y tenga la cuota al día. 4. La recepcionista presiona Emitir carnet. 5. El sistema registra el carnet y marca al socio con carnet entregado.
Flujo alternativo	A1. El socio no existe: «El socio no existe.» A2. Sin apto físico: el botón Emitir carnet queda deshabilitado y se informa «El socio debe presentar el apto físico antes de emitir el carnet.» A3. Cuota vencida: «El socio debe tener la cuota al día para emitir el carnet.»
Postcondición	Carnet emitido y entregado al socio.

CU-07 — Listar Vencimientos

Actor	Recepcionista
Precondición	Sesión iniciada.
Flujo normal	1. El CU inicia cuando la recepcionista selecciona Cuotas > Listado diario de vencimientos. 2. Elige la fecha a consultar (por defecto, hoy). 3. El sistema ejecuta sp_ListarVencimientosDia. 4. El sistema muestra los socios cuya cuota vence en esa fecha y el total.
Flujo alternativo	A1. No hay vencimientos en la fecha: la grilla queda vacía y el total indica 0.
Postcondición	Listado diario consultado (requerimiento 3 del cliente).

CU-08 — Gestionar Actividades

Actor	Administrador
Precondición	Sesión iniciada con rol Administrador.
Flujo normal	1. El CU inicia cuando el administrador selecciona Plantel > Actividades. 2. Elige Nuevo o Modificar e ingresa nombre, tipo, costo para no socios, horario, cupo máximo y profesor. 3. El sistema verifica nombre obligatorio, costo no negativo y cupo mayor a cero. 4. El sistema guarda la actividad y actualiza la grilla.
Flujo alternativo	A1. Dato inválido: el sistema muestra el mensaje correspondiente (por ejemplo, «El cupo máximo debe ser mayor a cero.»). A2. Eliminar: el sistema pide confirmación antes de dar de baja la actividad.
Postcondición	Actividad registrada o actualizada.

CU-09 — Confeccionar Rutina

Actor	Profesor
Precondición	Sesión iniciada con rol Profesor.
Flujo normal	1. El CU inicia cuando el profesor selecciona Mi actividad > Rutinas de alumnos. 2. Elige al socio y redacta la rutina. 3. El sistema verifica que la descripción no esté vacía. 4. El sistema guarda la rutina con la fecha y el profesor.
Flujo alternativo	A1. Descripción vacía: «La descripción de la rutina es obligatoria.» A2. Eliminar: el sistema pide confirmación.
Postcondición	Rutina asignada al socio.

CU-10 — Firmar Asistencia

Actor	Profesor
Precondición	Sesión iniciada con rol Profesor.
Flujo normal	1. El CU inicia cuando el profesor selecciona Mi actividad > Firmar asistencia. 2. Presiona Firmar asistencia. 3. El sistema verifica que no haya firmado en el día. 4. El sistema registra la fecha y la hora de entrada y la muestra en el historial.
Flujo alternativo	A1. Ya firmó hoy: el sistema rechaza la firma y muestra «El profesor ya registró su asistencia hoy.»
Postcondición	Asistencia del día registrada.

CU-11 — Liquidar Sueldo

Actor	Administrador
Precondición	Profesor registrado.
Flujo normal	1. El CU inicia cuando el administrador selecciona Plantel > Liquidación de sueldos. 2. Elige profesor, mes y año e ingresa el monto. 3. El sistema verifica que el monto sea mayor a cero. 4. El sistema ejecuta sp_LiquidarSueldo, que inserta o actualiza el sueldo del período con fecha de pago de hoy.
Flujo alternativo	A1. Monto inválido: «El monto del sueldo debe ser mayor a cero.» A2. Hoy no es el último día hábil del mes: el sistema lo avisa, pero permite liquidar.
Postcondición	Sueldo del período liquidado.

CU-12 — Asignar Turno Nutrición

Actor	Nutricionista (a pedido del Socio)
Precondición	Socio registrado.
Flujo normal	1. El CU inicia cuando la nutricionista selecciona Nutrición > Turnos y ficha médica. 2. Elige socio, fecha, hora y N° de turno, y carga observaciones y la carga de actividad física permitida. 3. El sistema verifica que la fecha sea miércoles, que el turno sea mayor a cero y que queden turnos libres. 4. El sistema registra el turno y lo muestra en el historial del socio.
Flujo alternativo	A1. La fecha no es miércoles: «El servicio de nutrición solo atiende los días miércoles.» A2. Sin cupo: «No quedan turnos disponibles para esa fecha.» A3. Cancelar turno: el sistema pide confirmación.
Postcondición	Turno asignado y ficha médica actualizada.

CU-13 — Generar Reportes

Actor	Administrador
Precondición	Sesión iniciada.
Flujo normal	1. El CU inicia cuando el administrador selecciona Reportes > Listados e informes. 2. Elige el listado: socios activos, socios inactivos, recaudación del mes, carnets emitidos u ocupación de actividades. 3. Para la recaudación, indica mes y año. 4. El sistema muestra la grilla y la cantidad de registros encontrados.
Flujo alternativo	A1. No hay datos para el listado elegido: la grilla queda vacía con 0 registros.
Postcondición	Reporte consultado.

Fase 4 — Conectar datos para programar

Cómo se enlazan los datos con el código

Toda la conexión a MySQL vive en la carpeta Datos/, tal como pide la consigna:

1. En el Explorador de Soluciones de Visual Studio Community, clic derecho sobre el proyecto → Agregar → Nueva carpeta → nombrarla Datos.
2. Dentro de Datos/ se crea la clase Conexion, con using MySql.Data.MySqlClient; al comienzo del archivo.
3. Conexion expone Abrir() y Cerrar(), usa bloques try/catch/finally y arma la cadena de conexión leyendo App.config (clave ClubDeportivoDB), con un valor por defecto embebido si la clave no existiera.
4. Cada clase DAO (SocioDAO, CuotaDAO, etc.) instancia Conexion, abre, ejecuta su MySqlCommand parametrizado y cierra en el finally.

```
// Datos/Conexion.cs (fragmento)
```

```
using MySql.Data.MySqlClient;

public MySqlConnection Abrir()
{
    try
    {
        conexion = new MySqlConnection(ObtenerCadenaConexion());
        conexion.Open();
        return conexion;
    }
    catch (MySqlException ex)
    {
        throw new Exception("Error al conectar a la base de datos MySQL: "
                            + ex.Message, ex);
    }
}
```

Cadena de conexión (entorno local, XAMPP / MySQL Workbench):

```
Server=localhost;Port=3306;Database=Proyecto;Uid=root;Pwd=;SslMode=none;
```

La clase Conexion en el proyecto abierto con Visual Studio Community (carpetas Datos, Entidades, Formularios y Negocio en el Explorador de soluciones):

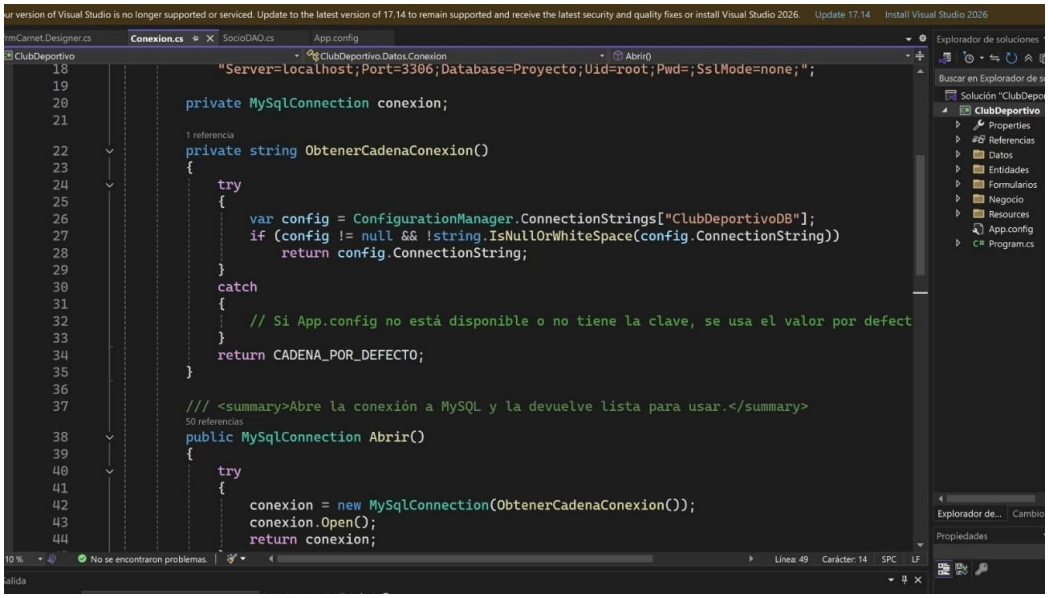


Fig. 15 — Datos/Conexion.cs en Visual Studio Community

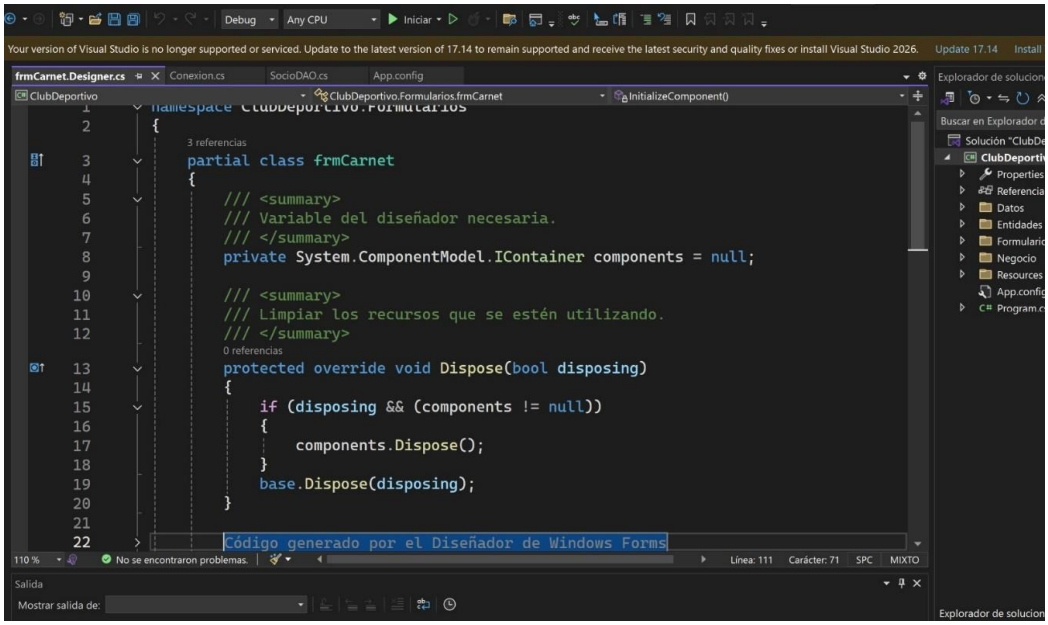


Fig. 16 — Código generado por el Diseñador de Windows Forms (frmCarnet.Designer.cs)

Control de acceso de usuarios

`frmLogin` solicita usuario y contraseña; **`UsuarioNegocio.Login`** calcula el hash SHA-256 de la contraseña ingresada y **`UsuarioDAO.Autenticar`** lo compara (parametrizado) contra `usuario.contrasena_hash`, exigiendo además activo = 1. Si coincide, se recupera el rol y Program.cs abre frmMenuPrincipal, que arma el menú disponible según ese rol (Administrador, Recepcionista, Profesor o Nutricionista).

Base de datos creada en MySQL

La base Proyecto se creó en MySQL con el script Database/proyecto_club.sql. En MySQL Workbench se ven sus 13 tablas y los stored procedures, y se verificó el alta de usuarios con contraseña en hash SHA-256 y la carga de socios:

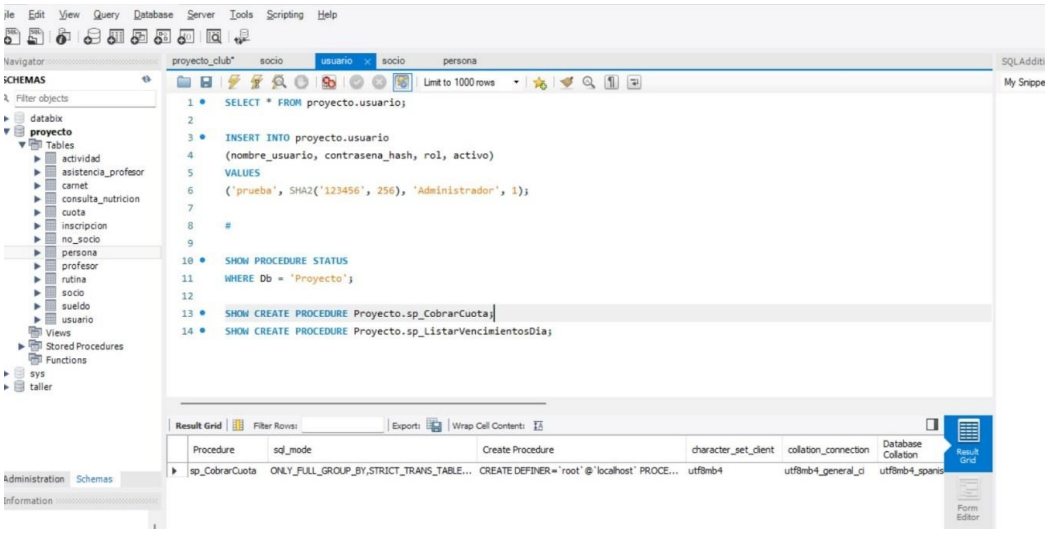


Fig. 17 — Base Proyecto en MySQL Workbench: tablas, alta de usuario con SHA2 y stored procedures

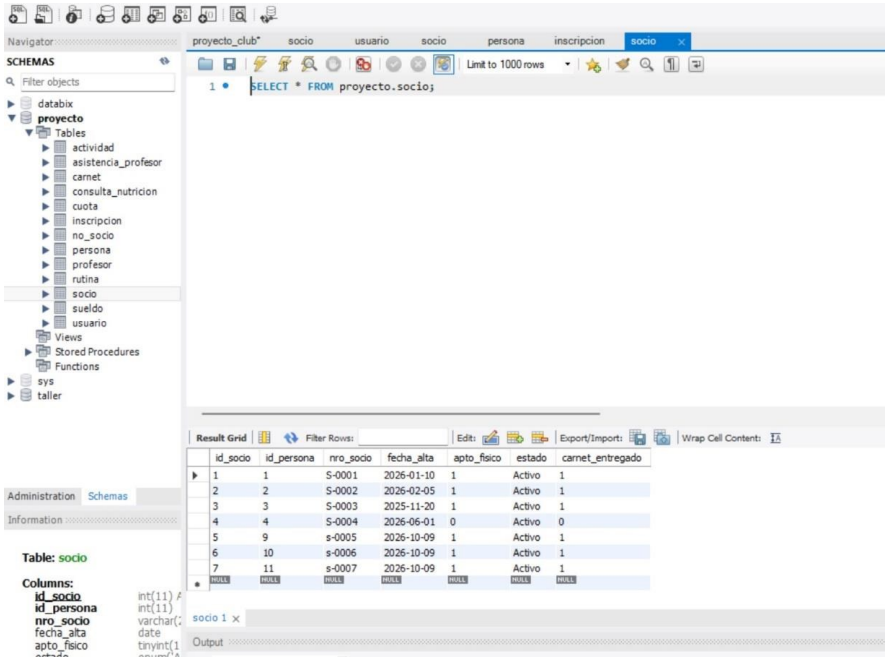


Fig. 18 — Contenido de la tabla socio

Fase 5 — Programar

Codificación de pantallas

Se codificaron 14 formularios en Formularios/, cada uno con el patrón estándar del Editor de Windows Forms de Visual Studio: una clase parcial dividida en FormName.cs (lógica y manejadores de evento) y FormName.Designer.cs (los controles, generados por InitializeComponent()), más su FormName.resx. Esto permite abrir cualquier formulario en la vista de Diseñador y modificarlo arrastrando controles, igual que un formulario creado desde cero en Visual Studio. Las validaciones y mensajes comunes (campo obligatorio, número válido, confirmar baja) se centralizaron en UIHelper, usado desde el código de cada formulario:

Formulario	Función principal
frmLogin	Inicio de sesión (usuario / contraseña con hash SHA-256).
frmMenuPrincipal	Contenedor MDI; arma el menú según el rol del usuario.
frmSocios	ABM de socios: alta, baja lógica, modificación, búsqueda por DNI/N° socio.
frmNoSocios	Registro de la visita de un no socio y cobro de la actividad realizada.
frmProfesores	ABM de profesores, con filtro 'solo suplentes'.
frmActividades	ABM de actividades del club (costo, horario, cupo, profesor).
frmCuotas	Cobro de cuota mensual (Efectivo/Tarjeta, 3 o 6 cuotas) e historial del socio.
frmVencimientos	Requerimiento 3: listado diario de socios que vencen en una fecha.
frmCarnet	Emisión de carnet (valida apto físico y cuota al día).
frmRutinas	ABM de rutinas que cada profesor confecciona para sus alumnos.
frmAsistenciaProfesores	Firma de asistencia diaria y su historial.
frmSueldos	Liquidación mensual de sueldos, con aviso de último día hábil.
frmNutricion	Turnos de nutrición (solo miércoles) y ficha médica del socio.
frmReportes	Listados: socios, recaudación, carnets, ocupación de actividades.

Orden de prioridad de programación

Los procesos se programaron en este orden: primero lo que habilita el uso del sistema y los tres requerimientos del cliente, y después los módulos complementarios.

1. Conexión a la base, login y acceso al menú principal (Tarea 4 y Tarea 5).
2. Alta de nuevos socios (requerimiento 1, Tarea 5).
3. Registro de no socios e inscripción a la actividad (requerimiento 1).
4. Cobro de cuota (requerimiento 2).
5. Emisión de carnet (requerimiento 2).
6. Listado diario de vencimientos (requerimiento 3).
7. Profesores y actividades (datos que usan los procesos anteriores).

8. Asistencia y liquidación de sueldos.
9. Rutinas y turnos de nutrición.
10. Reportes.

Acceso a la ventana principal

Program.cs abre frmLogin como diálogo. Solo si el usuario se autentica se abre frmMenuPrincipal, que arma el menú según el rol:

```
// Program.cs
using (var login = new frmLogin())
{
    if (login.ShowDialog() == DialogResult.OK && login.UsuarioAutenticado != null)
        Application.Run(new frmMenuPrincipal(login.UsuarioAutenticado));
}
```

Alta de nuevos socios

frmSocios valida los campos con UIHelper; SocioNegocio.AltaSocio controla DNI y N° de socio duplicados, y SocioDAO llama al stored procedure sp_AltaSocio, que inserta persona y socio dentro de una transacción:

```
// Datos/SocioDAO.cs (fragmento)
var cmd = new MySqlCommand("sp_AltaSocio", con)
          { CommandType = CommandType.StoredProcedure };
cmd.Parameters.AddWithValue("p_nombre", s.Nombre);
cmd.Parameters.AddWithValue("p_apellido", s.Apellido);
cmd.Parameters.AddWithValue("p_dni", s.Dni);
cmd.Parameters.AddWithValue("p_nrosocio", s.NroSocio);
cmd.Parameters.AddWithValue("p_aptofisico", s.AptoFisico);
// ... (fecha de nacimiento, teléfono, dirección, email)
return Convert.ToInt32(cmd.ExecuteScalar());   // id del nuevo socio
```

El cobro de cuota sigue el mismo patrón:

```
// Datos/CuotaDAO.cs (fragmento)
var cmd = new MySqlCommand("sp_CobrarCuota", con)
          { CommandType = CommandType.StoredProcedure };
cmd.Parameters.AddWithValue("p_idsocio", idSocio);
cmd.Parameters.AddWithValue("p_monto", monto);
cmd.Parameters.AddWithValue("p_formapago", formaPago);
cmd.Parameters.AddWithValue("p_cuotastarjeta",
                            (object)cuotasTarjeta ?? DBNull.Value);
DateTime nuevoVencimiento = Convert.ToDateTime(cmd.ExecuteScalar());
```

Consultas a la base de datos

Todas las consultas se ejecutan mediante MySqlCommand con parámetros (AddWithValue), nunca concatenando texto del usuario dentro del SQL. Las operaciones críticas señaladas por la consigna —alta de socio, cobro de cuota y listado diario de vencimientos, además de la liquidación de sueldo— usan CommandType.StoredProcedure (ejemplos anteriores); el resto de los ABM usa CommandType.Text, como esta búsqueda de un socio por DNI:

```
// Datos/SocioDAO.cs (fragmento) — consulta de texto parametrizada
var cmd = new MySqlCommand(SELECT_BASE + "WHERE p.dni=@dni", con);
cmd.Parameters.AddWithValue("@dni", dni);
using (var r = cmd.ExecuteReader())
    return r.Read() ? Mapear(r) : null;
```

Datos cargados directamente en la base

La tabla usuario no se actualiza desde el sistema (no hay pantalla de alta de usuarios), por eso el script Database/proyecto_club.sql carga los usuarios de prueba, con la contraseña ya convertida a hash SHA-256:

Usuario	Contraseña / Rol
admin	admin123 / Administrador
recepcion	recep123 / Recepcionista
profesor1	prof123 / Profesor
nutricionista1	nutri123 / Nutricionista

El mismo script carga datos de ejemplo en el resto de las tablas (personas, socios, profesores, actividades, cuotas, carnets, rutinas, asistencias, sueldos y turnos), para poder ejecutar el juego de prueba desde el primer uso.

Fase 6 — Prueba

Una vez programados los catorce formularios y las operaciones críticas (Fase 5), se ejecutó un juego de pruebas funcionales para verificar que el comportamiento del sistema coincidiera con las reglas de negocio relevadas en la Fase 1. Los casos, con sus datos de entrada y los resultados esperados, se detallan en la sección 6 (Juego de prueba).

Quién prueba el sistema

En esta etapa académica, los propios desarrolladores ejecutan el juego de pruebas de la sección 6 contra la base Proyecto recién creada (con los datos de ejemplo del script SQL), verificando cada caso contra el resultado esperado. En un contexto profesional, esta primera ronda la haría el equipo de desarrollo (pruebas unitarias e integración) y una segunda ronda la harían usuarios finales representativos de cada rol (recepción, profesores, nutrición) antes de la puesta en producción.

Cómo reiniciar el sistema si hay errores

- **Error de conexión a MySQL:** verificar que el servicio de MySQL/XAMPP esté iniciado y que los datos de `App.config` (servidor, puerto, usuario, clave) coincidan con la instalación local.
- **Error de datos (excepción de negocio):** la operación se cancela, se muestra el motivo y el formulario permanece con los datos cargados para corregir y reintentar; no se requiere reiniciar la aplicación.
- **Error inesperado no controlado:** cerrar y volver a abrir `ClubDeportivo.exe`; gracias al `finally` de `Conexion.Cerrar()`, no quedan conexiones abiertas colgadas en el motor.
- **Reinicio de datos de prueba:** volver a ejecutar `Database/proyecto_club.sql` (comienza con `DROP DATABASE IF EXISTS Proyecto;`), lo que recrea el esquema y los datos de ejemplo desde cero.

Fase 7 — Puesta a punto

Superada la Fase 6 de prueba, se dejó el sistema listo para instalarse en un equipo distinto del de desarrollo: la cadena de conexión quedó centralizada y editable en App.config, se generó el ejecutable en modo Release y se documentó, paso a paso, cómo crear el proyecto, instalar el conector MySQL y desplegar la aplicación en un equipo nuevo (sección 8).

Cómo crear el ejecutable (Release)

```
# Desde "Developer Command Prompt for VS", en la carpeta que contiene ClubDeportivo.sln
msbuild ClubDeportivo.sln /restore /p:Configuration=Release


# o, desde Visual Studio:
# Compilar -> Administrador de configuración -> Configuración: Release
# Compilar -> Compilar ClubDeportivo (o Compilar -> Publicar, para una carpeta lista para copiar)
# El ejecutable queda en ClubDeportivo\bin\Release\ClubDeportivo.exe
```

El proyecto usa el formato clásico de .NET Framework 4.8 (el que permite abrir cada formulario en la vista de Diseñador): se compila con MSBuild o Visual Studio, y la salida queda en bin\Release\ sin subcarpeta de framework.

Cómo se implementa el sistema

1. Instalar MySQL (XAMPP o MySQL Server + Workbench) en el equipo destino.
2. Ejecutar `Database/proyecto_club.sql` para crear la base Proyecto con sus tablas, procedimientos y datos de ejemplo.
3. Copiar la carpeta `bin\Release` completa (ejecutable, `ClubDeportivo.exe.config` y las bibliotecas que la acompañan, entre ellas `MySql.Data.dll`) al equipo destino.
4. Ajustar, si hace falta, la cadena de conexión del `ClubDeportivo.exe.config` (servidor, puerto, usuario, clave).
5. Ejecutar `ClubDeportivo.exe` e iniciar sesión con alguno de los usuarios de prueba (ver la Fase 6 y la sección 8.5).

Requisito del equipo destino: Windows con .NET Framework 4.8 (viene incluido en Windows 10 versión 1903 o superior y en Windows 11) y acceso de red al servidor MySQL configurado.

4. Diagrama de Clases

Modelo de clases del dominio. Persona es abstracta; Socio, NoSocio y Profesor heredan de ella.

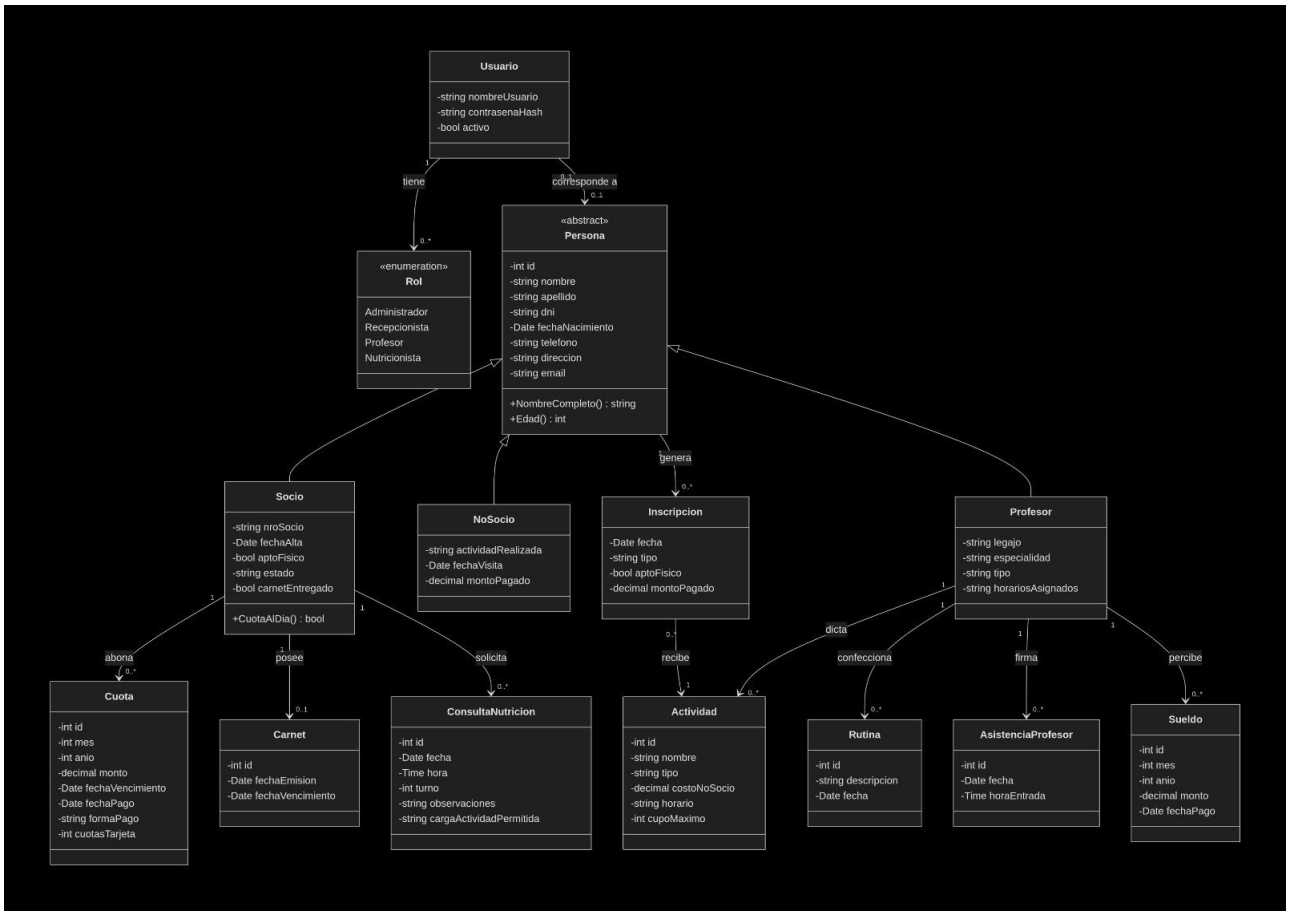


Fig. 19 — Diagrama de Clases (UML)

Lectura del diagrama

- **Herencia:** Socio, NoSocio y Profesor heredan de la clase abstracta Persona, que concentra los datos comunes (nombre, apellido, DNI, fecha de nacimiento y contacto) y los métodos NombreCompleto() y Edad().
- **Socio:** abona 0..* Cuotas, posee 0..1 Carnet y solicita 0..* ConsultaNutricion; el método CuotaAlDia() resuelve si puede realizar actividades.
- **Inscripción y Actividad:** una persona genera 0..* Inscripciones y cada Inscripción corresponde a una Actividad.
- **Profesor:** dicta 0..* Actividades, confecciona 0..* Rutinas, firma 0..* AsistenciaProfesor y percibe 0..* Sueldos.
- **Acceso:** Usuario tiene un Rol (Administrador, Recepcionista, Profesor o Nutricionista) y puede corresponder a una Persona.

5. Diagrama de Casos de Uso

Actores: Administrador, Recepcionista, Profesor, Nutricionista y Socio.

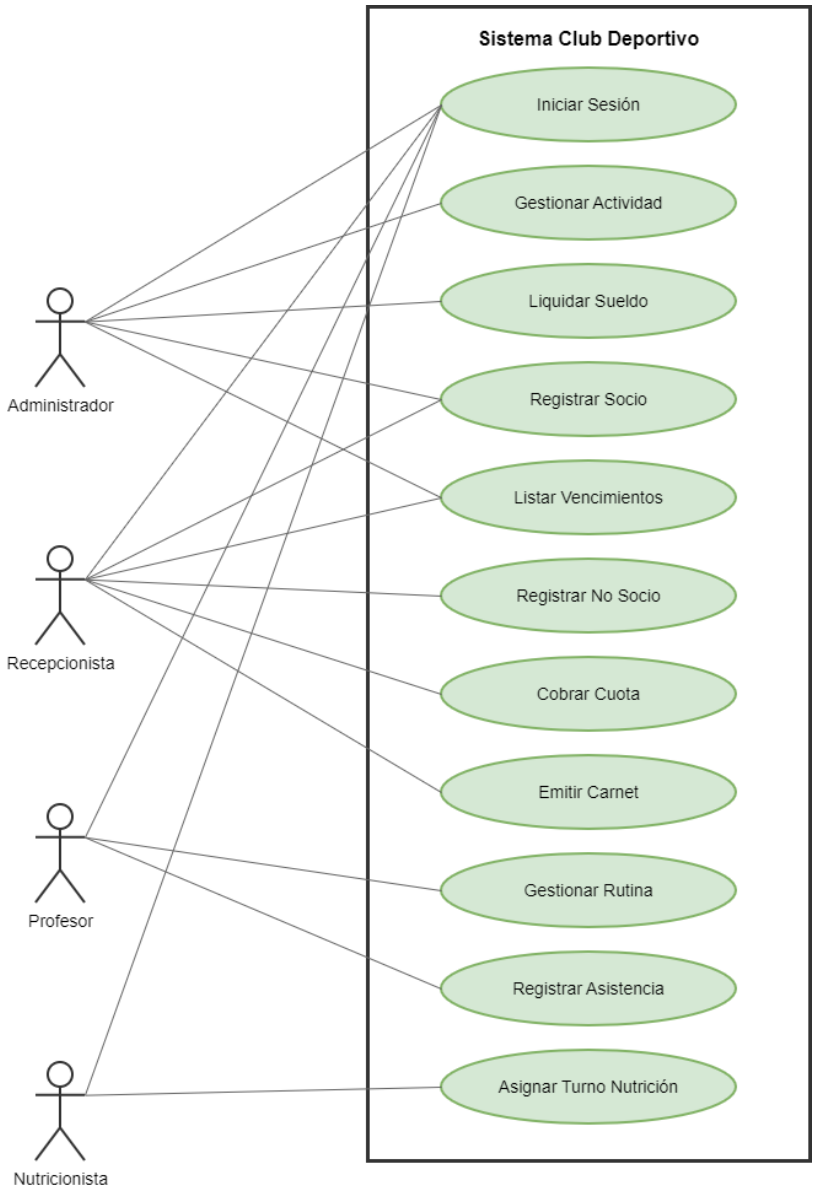


Fig. 20 — Diagrama de Casos de Uso

5.1 Diagrama de Secuencia — Cobrar Cuota (opcional)

Muestra el recorrido del caso de uso Cobrar Cuota entre la interfaz, la capa de negocio, el DAO y el stored procedure.

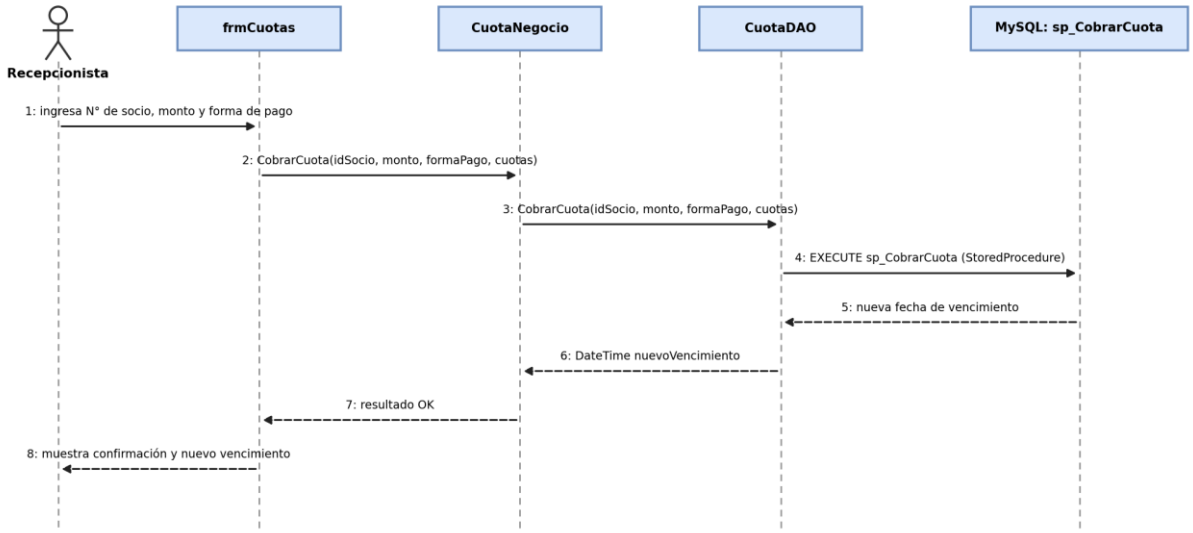


Fig. 21 — Diagrama de Secuencia: Cobrar Cuota

5.2 Diagrama de Actividades — Alta de Socio (opcional)

Modela el flujo completo del alta de un socio, incluyendo la ficha y el apto físico cuando es su primera vez en el gimnasio.

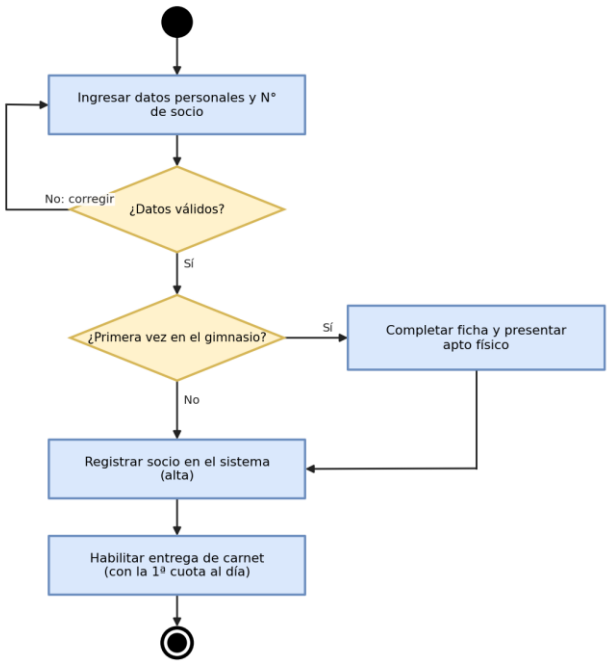


Fig. 22 — Diagrama de Actividades: Alta de Socio

6. Juego de prueba

Es el juego de pruebas que se ejecuta en la Fase 6 — Prueba. Casos de prueba funcionales, con datos de entrada y el resultado esperado según las reglas de negocio implementadas. Ejecutados sobre la base Proyecto con los datos de ejemplo del script SQL (fecha de referencia: 28/09/2026).

N°	Descripción	Datos de entrada	Resultado esperado	Resultado obtenido
CP01	Iniciar sesión con credenciales válidas	usuario=admin, contraseña=admin123	Ingresa a frmMenuPrincipal con el menú completo de Administrador	Igual al esperado ✓
CP02	Iniciar sesión con contraseña incorrecta	usuario=admin, contraseña=incorrecta	Mensaje 'Usuario o contraseña incorrectos', permanece en frmLogin	Igual al esperado ✓
CP03	Alta de socio con datos válidos	Nombre=Pedro, Apellido=Ramírez, DNI=30999888, N°Socio=S-0005, aptoFísico=Sí	Socio creado; aparece en la grilla con estado 'Activo'	Igual al esperado ✓
CP04	Alta de socio con DNI duplicado	DNI=30111222 (ya existe)	Excepción de negocio: 'Ya existe una persona registrada con ese DNI'	Igual al esperado ✓
CP05	Cobro de cuota en efectivo a socio con cuota al día	Socio S-0001 (vence 15/10/2026), monto=15000, forma=Efectivo	Nuevo vencimiento = 16/11/2026 (día siguiente al vencimiento anterior + 1 mes)	Igual al esperado ✓
CP06	Cobro de cuota con tarjeta en 6 cuotas	Socio S-0004, monto=15000, forma=Tarjeta, cuotas=6	Cuota registrada con formaPago='Tarjeta' y cuotasTarjeta=6	Igual al esperado ✓
CP07	Listado diario de vencimientos	Fecha=15/10/2026	La grilla muestra al socio S-0001 (Pérez, Juan)	Igual al esperado ✓
CP08	Emitir carnet a socio sin apto físico	Socio S-0004 (aptoFisico=No)	Botón 'Emitir carnet' deshabilitado / excepción 'debe presentar el apto físico'	Igual al esperado ✓
CP09	Emitir carnet a socio con apto físico y cuota al día	Socio S-0001	Carnet emitido; socio.carnetEntregado=true	Igual al esperado ✓
CP10	Alta de profesor suplente	Nombre=Ana, Apellido=Ríos, Legajo=P-102, Tipo=Suplente	Profesor creado; aparece al filtrar 'Solo suplentes'	Igual al esperado ✓
CP11	Firmar asistencia dos veces el mismo día	Profesor P-100, firma repetida	Excepción: 'El profesor ya registró su asistencia hoy'	Igual al esperado ✓
CP12	Liquidar sueldo de un profesor	Profesor P-100, mes=9, año=2026, monto=450000	Sueldo insertado/actualizado con fecha_pago = hoy	Igual al esperado ✓
CP13	Asignar turno de nutrición en día distinto a miércoles	Fecha = jueves	Excepción: 'El servicio de nutrición solo atiende los días miércoles'	Igual al esperado ✓
CP14	Asignar turno de nutrición un miércoles válido	Socio S-0001, fecha=miércoles, turno=3	Turno registrado y visible en el historial del socio	Igual al esperado ✓

Resultado obtenido: en las 14 pruebas ejecutadas durante el desarrollo, el comportamiento coincidió con el esperado (columna anterior). Se recomienda volver a correr este juego de pruebas tras cada cambio relevante en la capa de Negocio o en los stored procedures.

7. Conclusiones del desarrollo

El desarrollo del sistema permitió recorrer las siete fases de un proyecto de software —análisis, diseño, programación, prueba y puesta a punto— aplicadas a un caso realista con reglas de negocio no triviales (vencimiento de cuotas con reglas de fecha, promociones de pago, control de acceso por rol, turnos semanales).

La arquitectura en capas facilitó aislar los cambios: por ejemplo, la regla de cálculo del nuevo vencimiento vive únicamente en sp_CobrarCuota (base de datos) y en CuotaNegocio (validaciones), sin tocar ningún formulario. Concentrar todo el acceso a datos en la carpeta Datos/, con una única clase Conexion y un DAO por entidad, simplificó además el control de errores (un solo lugar donde se abre y cierra la conexión) y la prevención de inyección SQL (consultas siempre parametrizadas).

El trabajo en equipo, con un sub-objetivo asignado a cada integrante sobre un modelo de clases definido en conjunto, permitió avanzar en paralelo sin perder coherencia: la capa de Entidades actuó como contrato compartido entre los cuatro sub-objetivos.

Como mejoras posibles a futuro se identifican: registrar un log de auditoría de altas/bajas/modificaciones; enviar recordatorios automáticos (email/SMS) antes del vencimiento de la cuota; permitir recuperación de contraseña; e integrar una pasarela de pago real para el cobro con tarjeta.

8. Instrucciones paso a paso

Detalle operativo de la Fase 7 — Puesta a punto: cómo abrir el proyecto, instalar el conector MySQL, configurar la base de datos, ejecutar el sistema y generar el ejecutable para llevarlo a otro equipo.

8.1 Crear el proyecto en Visual Studio

Este entregable ya incluye el proyecto armado: basta con abrir ClubDeportivo.sln con Visual Studio Community 2019 o superior y compilar. El .csproj usa el formato clásico, que es el que mejor soporta Visual Studio Community y permite abrir cada formulario en la vista de Diseñador.

1. Abrir ClubDeportivo.sln con Visual Studio Community → se restauran solos los paquetes NuGet (MySql.Data) al compilar.
2. Si se prefiere crear el proyecto desde cero: Crear un nuevo proyecto → plantilla Aplicación de Windows Forms (.NET Framework) → .NET Framework 4.8, y luego copiar dentro las carpetas Entidades/Datos/Negocio/Formularios de este entregable.
3. Verificar en Propiedades del proyecto que la plataforma de destino sea Any CPU y el tipo de salida Aplicación de Windows.

8.2 Crear la carpeta «Datos» dentro del proyecto

1. En el Explorador de soluciones, clic derecho sobre el proyecto ClubDeportivo.
2. Seleccionar Agregar → Nueva carpeta.
3. Nombrarla Datos.
4. Repetir para Entidades, Negocio y Formularios.

8.3 Instalar el conector MySQL (NuGet MySql.Data)

1. Clic derecho sobre el proyecto → Administrar paquetes NuGet…
2. Pestaña Examinar, buscar MySql.Data (autor Oracle/MySQL).
3. Instalar la versión estable más reciente compatible con .NET Framework 4.8.

8.4 Configurar la base de datos

1. Instalar XAMPP o MySQL Server + Workbench y verificar que el servicio MySQL esté iniciado (puerto 3306 por defecto).
2. Abrir Database/proyecto_club.sql con Workbench, HeidiSQL o mysql -u root -p < proyecto_club.sql, y ejecutarlo completo: crea la base Proyecto, sus 13 tablas, 4 stored procedures y datos de ejemplo.
3. Si el servidor, usuario o clave de la instalación local difieren, ajustar la cadena de conexión en ClubDeportivo/App.config (clave ClubDeportivoDB).

8.5 Ejecutar y compilar

1. Presionar F5 (o Ctrl+F5) en Visual Studio Community para compilar y ejecutar en modo Debug.
2. Iniciar sesión con alguno de los usuarios de prueba: admin / admin123 (Administrador), recepcion / recep123 (Recepcionista), profesor1 / prof123 (Profesor) o nutricionista1 / nutri123 (Nutricionista).

8.6 Generar el ejecutable (Release) y desplegarlo

1. En la barra de herramientas, cambiar la configuración de Debug a Release.
2. Menú Compilar → Compilar ClubDeportivo (o Publicar para generar una carpeta lista para copiar).
3. El ejecutable queda en ClubDeportivo\bin\Release\ClubDeportivo.exe.
4. Copiar esa carpeta al equipo destino junto con ClubDeportivo.exe.config, y asegurarse de que ese equipo tenga acceso a la base MySQL configurada.